NOVA IMS INFORMATION MANAGEMENT SCHOOL

Exercises Book

Computation I

Your extra exercises and study guide ☺

NOVA IMS

2026

Contents

1 About this Exercises Book **1**

2 Expressions, Data types and Variables **1**

2.1 Variables and Data types 1

2.1.1 Variable names (identifiers) 2

2.2 Expressions 2

2.2.1 Arithmetic Expressions 2

2.2.2 Relational Expressions 3

2.2.3 Logical Expressions 3

2.2.4 Variable update 3

2.3 Exercises 4

3 Introductory String operations and math library **6**

3.1 Basic String Operations 6

3.2 Type conversions 7

3.3 math module (library) 7

3.4 Exercises 7

4 User input: interaction with the user **9**

4.1 The input function 9

4.2 Exercises 10

5 Lists: Part I **12**

5.1 List Operations 13

5.2 Exercises 13

6 Conditionals: if, elif and else **14**

6.1 Conditional instruction If - One way 14

6.2 Conditional instruction If - Two way 15

6.3 If, elif, else 16

6.4 Exercises 17

7 Loops: While and for loops **21**

7.1 While loops 21

7.2 Obtaining numerical input from the user: the is.digit() function 23

7.2.1 the is.digit() function 23

7.2.2 Detecting an integer, a float and string/bool 23

7.3 Assuring user input is numerical 24

7.4 For loops 24

7.5 Exercises 27

8 Lists: Part II **29**

8.1 Traversing Lists 29

8.2 List Comprehensions 30

8.3 Exercises 30

9 Matrices and nested for loops **33**

9.1 Matrices 33

9.2 A more in depth look into Nested for loops 34

9.3 Exercises 36

10 Functions **39**

10.1 Function creation and calling 39

10.2 Function rules 41

10.3 Exercises 42

1 About this Exercises Book

This Exercises Book will serve as your friend and guide throughout your Computation I path.

Simply reading this book and solving its exercises will **not** be enough for you to pass the class. This book serves as complementary material for your studying and therefore is a complement to the other materials provided to you and, **most importantly, the classes**.

The guiding materials in the book are **very** summarized and do not contain all the information needed for the understanding of the subjects at hand.

The exercises in each section are **not** ordered from easiest to hardest. Thus, if you get stuck in an exercise do not fear continuing.

The goal of this exercise book is to allow you to practice coding and quickly review some core concepts. If for any reason you have already solved an exercise proposed in this book (for instance in class) do not skip the exercise and do not copy the solution from what you previously did! Try to solve each exercise from scratch to make sure you understand the concepts and are able to solve things on your own!

Good studying and good luck ☺

2 Expressions, Data types and Variables

2.1 Variables and Data types

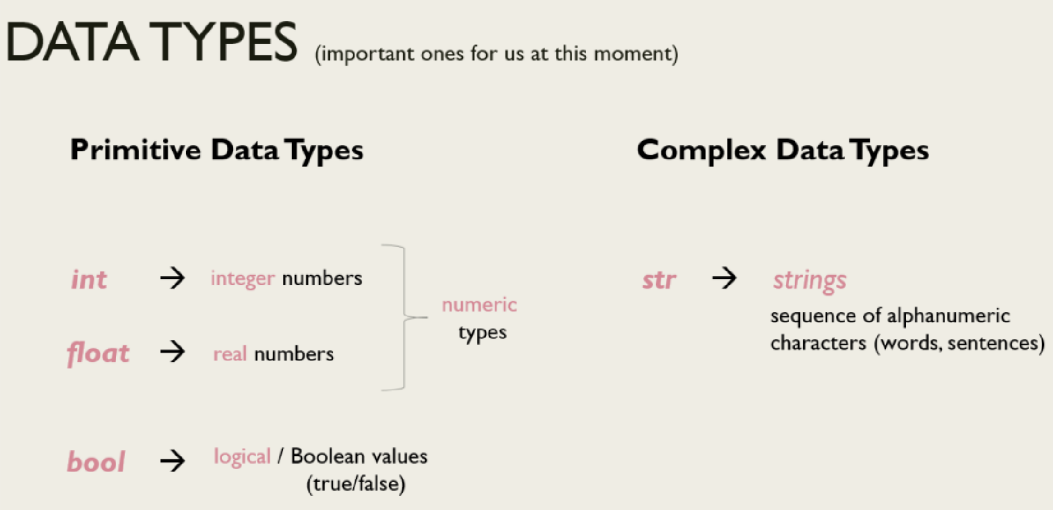
- A variable is identified by a name (identifier), a data type and a value.

  <name> = <value>

  For example:

  age = 42

- You do not need to specify the data type, Python knows this information on its own!



- Remember...
  - The value of a variable of type string must be between quotes (i.e., "Gertrude" or 'Gerturde')

2.1.1 Variable names (identifiers)

The name of a variable is a sequence of characters where:

- The first character cannot be a number
- Only lowercase, uppercase, number and underscore are allowed (no space, arithmetic character, punctuation sign...)
- Keywords cannot be used as identifiers. (e.g., and, or, else, True, False)

2.2 Expressions

2.2.1 Arithmetic Expressions

- A + B $\Longrightarrow$ + is the sum of A and B
- A - B $\Longrightarrow$ - is the subtraction of A by B
- A * B $\Longrightarrow$ * is the multiplication of A and B
- A ** B $\Longrightarrow$ ** is the power of A to the power of B
- A / B $\Longrightarrow$ / is the division of A by B

- A / B $\Longrightarrow$ / is the division of A by B
- A // B $\Longrightarrow$ // is the floor division of A by B
- A % B $\Longrightarrow$ % is the remainder of the integer division of A by B

2.2.2 Relational Expressions

Relational Expressions take the value of either True or False.

- A == B $\Longrightarrow$ == represents the equality operator
- A != B $\Longrightarrow$!= represents the differences operator
- A >= B $\Longrightarrow$ >= represents the superior to or equal to operator
- A <= B $\Longrightarrow$ ** <= represents the less than or equal to operator
- A > B $\Longrightarrow$ > represents the superior to operator
- A < B $\Longrightarrow$ < represents the inferior to operator

2.2.3 Logical Expressions

Logical Expressions take the value of either True or False.

- not(A) $\Longrightarrow$ == NOT operator (reverse the result)
- A and B $\Longrightarrow$ AND operator (True if both statements are True)
- A or B $\Longrightarrow$ >= OR operator (True if one of the statements is True)

2.2.4 Variable update

- x += 2 $\Longrightarrow$ means x = x + 2
- x -= 2 $\Longrightarrow$ means x = x - 2
- x *= 2 $\Longrightarrow$ means x = x * 2
- ...

2.3 Exercises

The exercises here are not supposed to be solved with ifs yet! Just print the answer of the expression (i.e., True or False). Of course if you already know ifs, you may use them!

1. What are the final results of x after the execution of the following blocks of code?

   (a) x = 10.0
   x = x - (x/100)*10
   x = x + (x/100)*10
   x *= 1

   (b) a = 1
   b = 2
   c = 4
   d = 6
   e = a*c+1
   x = ((a+b+c+d+e < 3*a+2*b-3*c) and d/c < 1) and False

2. Write a program that contains and prints the results of the following tests:

   (a) test 1 - Verify that the sum of two variables, a and b, with values of your choice, is larger than 100.

   (b) test 2 - Verify that the length of a string s, defined by you, has a length between 5 and 10.

   (c) test 3 - Verify the result of the expression (test1 or test2).

3. Write a program that outputs the result of the expression $(x + y)^2$ where x and y are integer numbers of your choice.

4. Write a program that calculates the volume of a sphere given a radius r of your choice. TIP: Sphere volume = $(4 \times \pi \times r^3)/3$

5. Create a variable age with an age of your choice. Verify if five years ago a person with that age would be considered a minor (i.e., under 18).

6. Create a variable salary with a value of your choice. Verify if a person earning twice as much per month earns more than 3000 a month.

7. Create a variable price (that represents the price of the Madden 24 video-game on steam) with the value of 69.99. Verify how much the game would cost without taxes. To solve this exercise, assume a tax rate of 23%.

8. Create three variables with numbers of your choice. Verify if the first number is bigger than the sum of the remaining two values.

9. Create a variable x with a value of your choice. Your professor would be happy if the variable you chose is even, not divisible by 5 and bigger than 10. Print on console if your professor is happy with the variable you chose.

10. Create two integer variables y and z, with values of your choice. Your professor would be happy if the y is between 21 and 40 or if z is more than 300 or is 74. Print on console if your professor is happy with the variables you chose.

11. Create an integer c with a value of your choice. The professor is only happy if c is either 1, 2 or 3. Print on console if your professor is happy.

12. Create three integer variables a, b, and c with values of your choice. Verify if the value of a squared is greater than the product of b and c.

13. Define two variables, *height* and *weight*, with values of your choice. Calculate and verify if the BMI (Body Mass Index) is greater than 25. (BMI is calculated as $\frac{\text{weight}}{\text{height}^2}$).

14. Create a variable *balance* with a value of your choice. Check if, after adding a bonus of 10% of the balance, the total amount exceeds 200.

15. Define a variable *temperature* with a value of your choice. Verify if, after increasing the temperature by 20% of its current value, it exceeds 37 degrees (body temperature).

16. Create two variables, *length* and *width*, with values of your choice. Verify if both *length* is greater than 10 and *width* is greater than 5. Print the result of this check.

17. Create three variables, a, b, and c, with values of your choice. Verify if $a + b$ is greater than 50 or if c is less than or equal to 20. Print the result of this check.

18. What are the final results of y after the execution of the following blocks of code?

    (a) y = 5.0
    y = y + (y/10)*2
    y = y - 3
    y *= 2

    (b) m = 3
    n = 7
    p = m * 2
    q = n - 3
    y = (m + n + p > q * 2) or (n / m > 1)

    (c) x = 8
    y = 3
    z = 5
    y = (x % z == 3) and (z * y < x)

3 Introductory String operations and math library

3.1 Basic String Operations

- len(A) $\Longrightarrow$ length of A (i.e., how many characters in A)
- A[i] $\Longrightarrow$ character of the string A in position i (remembering that the first position starts at 0).
- A[i:k] $\Longrightarrow$ substring. i included, k excluded
- A + B $\Longrightarrow$ concatenation of A and B
- A * X $\Longrightarrow$ repeating string A, X times
- A.lower(), A.upper() $\Longrightarrow$ changing to lower and upper case
- A.replace(old,new) $\Longrightarrow$ // replacing old with new
- B in A $\Longrightarrow$ % check if A contains B

3.2 Type conversions

Strings that only contain numbers can be converted into different variable types in the following way:

- int(a) $\Longrightarrow$ returns the string a in an int like form
- float(a) $\Longrightarrow$ returns the string a as a float
- str(a) $\Longrightarrow$ returns the number a in a string like form

3.3 math module (library)

The math library must be imported by writing:

import math

Then we can make use of its functions, such as:

- math.pi $\Longrightarrow$ mathematical pi
- math.pow(c, d) $\Longrightarrow$ c to the power of d
- math.sqrt(a) $\Longrightarrow$ square root of a
- math.cos(a) $\Longrightarrow$ cosine of a

3.4 Exercises

1. Using the math library, write a program that outputs the result of the expression $(x + y)^2$ where x and y are integer numbers of your choice.

2. Create a variable that saves a sentence of your choice with the length of at least 7. Create the following variables:

   - The length of the sentence
   - The 6th character of the sentence
   - The prior to last character of the sentence.

   Print in the console the value of the variables.

3. Create a string called name, with a name of your choice. Verify if the first letter of the name is an 'A'

4. Create a string called number with the value "74". Turn the string into the correct variable type and calculate its square root.

5. Create a string with a value of your choice. Assess if the size of the string is divisible by 3. TIP: you can use modulus (%) to determine if a number is divisible by another, can you remember how?

6. Create a string with at least two words, of your choice. Determine if the first letter of the string is the same as the first.

7. Create two variables a, and b with values of your choice. See if $(a + b) \times 2/3^2$ is bigger than 100.

8. Create a string called st with the value "I burnt my dinner". Remove all the spaces from the string. Then, replace all the vowels in the string with "xx". Finally, print the resulting string as well as its size.

9. Create two strings a and b, with values of your choice. First determine if string b contains string a. Then, print the size of the two strings combined.

10. Create a string a with the value "I believe in magic" and another string with the value "(the gathering)". Create a string that's the result of combining these two strings together. Then, print the resulting string a number of times that corresponds to the size of the second string (i.e, "(the gathering)").

11. What is printed on console after this code is executed?

```
a = "188"
b = "74"
c = a + b * 2 + str(int(a) + 2)
```

12. What is printed on console after this code is executed?

```
a = 84
b = "22"

c = int(a + b) + int(b[0])
```

13. Create a string called *text* with the value "Learning Python is fun!". Extract and print the substring that contains the first 8 characters, then print the substring that contains the last 4 characters.

14. Create a variable *phrase* with a sentence of your choice. Create two additional variables: one that converts all letters in *phrase* to uppercase and another that converts all letters to lowercase. Print both variables.

15. Define two strings, a and b, with any values of your choice. Using string concatenation, create a new string that repeats string a three times, followed by string b two times. Print the resulting string.

16. Define two integer variables, p and q, with values of your choice. Using the math library, calculate the logarithm of p and raise q to the power of p. Print both results. TIP: Use `math.log()` and `math.pow()` for these calculations.

17. What is printed on the console after this code is executed?

```
a = "56"
b = "12"
c = str(int(a) * 2) + b[1] + a[0]
```

18. What is printed on the console after this code is executed?

```
x = "150"
y = "75"
z = int(x[1]) + int(y[0]) + len(x + y)
```

4 User input: interaction with the user

4.1 The input function

We can obtain input from the user by using the input() function. This function receives as parameter the string that we want to have printed on console during the input request.

For example:

```
input("What is your age")
```

will print "What is your age" on console while allowing the user to type in console to respond. We can save the user's input into a variable like this:

age = input("What is your age")

Note that **all the input** that is inserted by the user is saved in a **string** type variable. Thus, if we want to store it as a different type of variable we must **convert** it into the right format. For example:

age = int(input("what is your age")).

So...

- input("What is your age?") $\Longrightarrow$ Prints "What is your age?" on console, allowing the user to respond but not storing that response.

- age = input("What is your age?") $\Longrightarrow$ Prints "What is your age?" on console, allowing the user to respond and stores a variable called age with the value of the input in string-like form (e.g., "18").

- age = int(input("What is your age?")) $\Longrightarrow$ Prints "What is your age?" on console, allowing the user to respond and stores a variable called age with the value of the input in an int form (e.g., 18).

4.2 Exercises

1. Write a program that asks the user for two words and prints "True" if the first word is bigger than the other one and "False" otherwise (in other words. Check if the first word is bigger than the second one)

2. Create a program that asks the user for their age. How old will the user be in 10 years? What about 15 years?

3. Ask the user to write a sentence. Then, ask them what letter of that sentence they want to print (i.e., if they insert 1 they want to print the first letter, 2 the second, etc...). Print that letter on console.

4. Ask the user to insert their name. Check if their name has an even number of letters or not. (If the name has an even number of letters True should be printed on console. False should be printed otherwise)

5. Ask the user for their student number. Print their NOVAIMS student e-mail (assuming they are bachelor degree students) on console. Note that if the student number is 20217416 the email is r20217416@novaims.unl.pt

6. Ask the user to insert two numbers of their choice. Is the first number divisible by the second one?

7. Create a program that asks the user for their age. Then, the program prints False on console if the user is a minor and True otherwise.

8. Create a program that asks the user for the price of the game they would like to purchase. Then print on console how much that game would cost with a 10% discount, a 15% discount and a 20% discount.

9. Create a program that asks the user for their name. Then, ask the user for their age. Print True on console if the size of the user's name is divisible by their age / 4.

10. Create a program that asks the user for a string of their choice. Then ask the user to insert a number from 0 until the size of the string / 2. Then ask the user to insert a number from the size of the string / 2 until its size - 1. Print on console the substring from the first number to the second number inserted.

11. Ask the user to insert a sentence. Check if the team Benfica or the Sporting is mentioned in the sentence.

12. Ask the user to enter their favorite color and their favorite animal. Print "True" if the color has more letters than the animal, and "False" otherwise.

13. Create a program that asks the user for their birth year. Then, calculate and print how old they will be in 5, 20, and 50 years.

14. Ask the user to write a sentence of their choice. Then ask for a position number. Print on the console the letter that is located at that position in the sentence. (For example, if they input 3, print the third letter in the sentence.)

15. Create a program that asks the user for the year they started school. Print in what year they will graduate assuming their degree is 3 years.

16. Ask the user to insert a date in the format MM/YYYY/DD. Create a new variable called correct which contains the date in the format DD/MM/YYYY. Print it.

17. Ask the user to insert two grades. Then, ask the user how much % each one of their grades is worth for their final grade. Then, calculate the final grade.

5 Lists: Part I

- A List is a data type in Python that can store sequences of objects (i.e. any other values or expressions that can be stored in normal variables).

- Lists are typically associated with [ ] (square brackets), which are used both for creating them and accessing its elements. A list can be created by writing its contents, separated by commas, and surrounded by brackets, such as:

```
my_list = [1, 2, "apple", True]
```

- Lists can contain any number of elements from any data type, and multiple types can be present in the same list

- Each element of a list is associated with a specific **index**. The first item is index 0, the second is index 1, the third index 2, and so on and so forth. Negative indexes can also be used to start from the last element, so index -1 is the last element of the list, -2 is the second to last, etc.... It is important to note that the last index of a list is always the length - 1.

```
print(my_list[1])
print(my_list[-1])
# Output:
>>> 2
>>> True
```

- Both the elements and size of a list can be changed. We can add an element to a list with the append function, and change another by accessing its index and assigning it another value (much like when creating a variable).

```
my_list.append(4.0)
my_list[0] = 10
```

5.1 List Operations

- lt.append(a) $\Longrightarrow$ adds (appends) the value of a to the list.
- lt1 + lt2 $\Longrightarrow$ returns a list that contains the elements of lt1, followed by the elements of lt2
- lt * 2 $\Longrightarrow$ returns a list with the values of lt repeated twice
- lt.pop() $\Longrightarrow$ removes the last value of the list
- a in lt $\Longrightarrow$ returns True if the value of a is in lt, and False otherwise
- len(lt) $\Longrightarrow$ returns the size of the list (i.e., the number of values it contains)

5.2 Exercises

1. Create a list of integer numbers of your choice. Ask the user for their age. Add the age to the list and print the result.

2. Create a list of your choice. Replace the first value with "LOML".

3. Create a list of your choice. Ask the user for their favourite word. Add the word to the list. Then, add to the list the size of the list. So, for example. [1, 2, "Words"] would become [1, 2, "Words", 3].

4. Create a list of your choice. Change the middle point of the list (calculated based on the list size) to the value that's the last element of the list.

5. Create a list of your choice. Is the size of the list even? Ask the user to insert a sentence. Does adding the size of the sentence to the size of the list result in a number that's bigger than 40 or even?

6. Ask the user for their student number. Make use of a list to find out if the student number belongs to a student in the honour board. The honour board has 4 students: 20246849, 20248688, 20241234, 20244444.

7. Create a list of your choice. Ask the user for two numbers. Add the two numbers to the list without making use of the .append() function

8. Ask the user for two sentences of at least 10 characters each. Create a list that contains the first 4 letters of the first sentence and the last 4 letters of the second one.

9. Create a list of your choice. Print on console the list and ask the user to choose the index of the value they would like to get rid of. Delete from the list the value that is in the chosen index.

10. Create a list of strings your choice. Add to the list the size of the first string. Then, remove the second string from the list.

11. Create a list of at least 4 different words of your choice. Ask the user for their favorite word. Insert this word into the middle of the list (calculated based on the list's size). Print the updated list.

6 Conditionals: if, elif and else

6.1 Conditional instruction If - One way

- When do we use it? $\Longrightarrow$ When we want to execute a command/command block only if a certain condition is satisfied.

- How do we use it? $\Longrightarrow$ Using the following code, where the boolean expression represents the previously mentioned condition:

```
if <Boolean expression>:
    <block1>
```

- How does it work? - Step by step:

1. The boolean expression is evaluated.

2. If the boolean expression is true (if it has the value True), Then the commands in <block1> are executed

3. otherwise (if the boolean expression has the value False), the program continues, ignoring the command blocks in the corpus of the if (i.e., ignoring <block1>).

6.2 Conditional instruction If - Two way

- When do we use it? $\Longrightarrow$ When the execution of a command/command block depends on the true or false value of a certain condition.

- How do we use it? $\Longrightarrow$ In the following way:

```
if  <Boolean expression>:
      <block1>
else:
      <block2>
```

- How does it work? - Step by step:

1. The boolean expression is evaluated.

2. If the boolean expression is true (if it has the value True), Then the commands that belong to the corpus of the if are executed (i.e., <block1>).

3. Otherwise (if the boolean expression is False), then the command/command block in the corpus of the else are executed (i.e., <block2>).

Remember that the **else never has a boolean expression!** It is what happens if the if expression didn't yield True, so we do not need to state any condition!

6.3 If, elif, else

- When do we use it? $\Longrightarrow$ When we have more than two possible conditions.

- How do we use it? $\Longrightarrow$ In the following way:

```
if <Boolean expression1>:
      <block1>
elif <Boolean expression2>:
      <block2>
…
elif <Boolean expressionN>:
      <blockN>
else:
      <blockN+1>
```

- How does it work? - Step by step:

  1. The boolean expression is evaluated.

  2. If the boolean expression 1 is true (has the value True) than command block 1 is executed.

  3. If the boolean expression 1 is false (has the value False):

     (a) If the boolean expression 2 is true (has the value true) than command block 2 is executed.

     (b) If the boolean expression 2 is false then the next boolean condition is evaluated..

(c) And so on and so forth... until one of the elif conditions yields True (and its corresponding code block gets exercuted) or they were all tested and none of them yielded True (in which case the else block of instructions is executed)

Remember...

- We can only execute one command block!!
- We can have as many elif as we want!
- We must always have a : after an if, elif or else
- The identation in Python means "this block of code that is idented belongs to the instruction that is on top of it". Thus, make sure you ident all the instructions that belong to an if! Then, write the elif on the same level as the if and ident all of the content of the elif below the elif and so on and so fourth!

6.4 Exercises

1. Write a program that asks the user for a number. If the number is positive and even, print "Nice and pretty" on console. If the number is positive but odd complain that it bugs you. Otherwise, inform the user that you want positive vibes in your code.

2. Write a program that verifies if a year is a leap year or not. The year must be an integer number (superior to 0) that is defined by the user. If the value inserted is less than 1, the program should print "sorry I cannot work like this".

   Note: A year is a leap year when it is divisible by 400 or when it is divisible by 4 and not by 100.

3. Write a program that:

   - Asks the user for the diameter of a circle, d and saves it in a variable.
   - if the value inserted by the user is bigger than 0, calculate the perimeter of the circle.
   - if the value inserted by the user is inferior to zero, print an error message.

- if the diameter is 0 the perimeter is 0

  Note: diameter $= 2 \times radius$ and perimeter $= 2 \times \pi \times r$

4. Write a program that receives the salary that three employees receive, per hour, in a company. This program should calculate the total payment the employees receive at the end of the month, knowing that they work 8h a day, 5 days a week. Consider that this month begins on a Monday and has 31 days.

   - If the final value is bigger or equal to 10.000 ethe program returns a message saying "We are being too nice, we need to lower the salaries"

   - If the final value is less than 10.000 and bigger or equal to 5.000 e, the program should return a message saying "We are fair bosses, yey us!"

   - If the final value is 5.000 or less, the program returns a message saying, "We need to increase the salaries otherwise nobody will want to work here!"

     TIP: There are 23 workdays in months that begin on a Monday.

5. Ask the user how much mini-test 1 is worth for their grade. Then ask the same about mini-test2 and the final exam. Proceed to ask the user their grades in the 3 evaluation moments. Then, inform the user of their final grade. If that grade is greater than 18, congradulate them on it!

6. Ask the user for their phone number, promising them you're not going to be weird about it. Then, if their phone number has more than 9 digits inform the user that they come from a large country. If it has between 8 and 9 inform the user that they come from a medium-small country. Finally, if they have less than 8 digits, inform the user that they come from a small country.

7. Write a program that receives 4 numbers from the user. Save the numbers in variables named x, y, z and w, respectively. Create a variable called min of the correct datatype, that saves the minimum value from the inserted numbers. Calculate min using only conditional ifs and print its value in console.

   TIP: If you know anything about cycles you can solve the exercise using them! If not, just solve it using ifs as previously requested!

8. Write a program that asks the user for their full name. If their name has more than 15 letters print "Damn... That is a long long name". If the name has between 10 to 14 letters say "yeah, I expected something like this...". Otherwise, say "This name

is tiny hehe".

9. Create a program that asks to insert their full address. Then, if the user is from Lisbon print "You are a little lettuce" on console. If not say "ooo Interesting!"

10. Create a program that asks the user to insert two numbers, a and b. Then if the sum of the two numbers yields an even number print "I am proud of this". If not, print "I am not ok with this".

11. Ask the user how much the video-game they want to buy costs. If the the video-game with a 15% discount would be cheaper than 12 euros, tell the user to buy the game right now. If the price is between 12 and 40, tell the user that they should buy the game only if they really really like it. If the price is higher than that tell the user that unless it's their dream game, the price is not worth it

12. Ask the user to insert the values for two variables, c1 and c2. These variables correspond to the sides of a triangle. Then, if the resulting hypotenuse is bigger than 20 print "That is a big triangle"! If its smaller than 20 but bigger than 10 print "Nice triangle!". Otherwise, print "That's smol".

13. Ask the user how many days of delay they have in returning their library books. Then, ask the user how many books are with delay. Calculate and inform the user how much they have to pay in delay fees, knowing that:

    - Each book that is delayed costs 2 euros per day of delay
    - An additional fee of 16 euros is added to the total fine if the delay is of more than 30 days.
    - If more than 3 books are delayed for more than 10 days there is an extra 10 euros fee added to the fine.
    - The total fine to be paid cannot exceed 140 euros.

14. Ask the user what is the name of the dish they want to prepare. Then, ask them how long they are going to take to prepare it. If the dish contains beef or chicken and the preparation time is less than 30 minutes print "That is highly suspicious" on console. If the dish has the word vegetarian in it and takes more than 45 minutes to cook, inform the user that "The dish seems to take too long for what it's worth". Additionally, If the dish contains pork, inform the user that it is not Kosher.

15. Ask the user to describe in a sentence why they love their hobby. If the word happiness is in the description, inform the user that they are optimistic and jolly. Also, check if replacing the word happiness with the word sorrow yields a string that is smaller than 16 characters. If so, tell the user that "a sadder answer would be short". Finally, if the user did not use hapiness or joy to describe their hobby, ask them if they are sure they actually like it.

16. Ask the user for their name. If their name ends with 'A', print their name 10 times. If it ends with an 'E', print the first 2 letters of their name 4 times. If it starts with a vowel, print the last two latters of their name 4 times. Otherwise, print their name once.

17. Ask the user for a number. If the number has more than four digits, print the result of dividing the number by 2. If it has four digits or less, print the results of multiplying the number by 1.5.

18. Ask the user to insert the cost of the book they are trying to purchase. Then, ask the user to insert 1 if it's a book, 2 if it's a comic book and 3 if its a study manual. If they insert anything other than these options, inform them they are silly. Print the final cost of the purchase knowing that books have a 5% discount, comic books a 10% discount and study manuals a 20% discount.

19. Ask the user for their age and current money in their wallet. Then, ask the user how many cocktails they want. Know that each cocktail costs 8 euros and the bar currently has 10 cocktails left. If the user is of age (older than 18) and can afford the total cost of the cocktails inform them that you will sell it to them, but only if the bar has enough. If not, inform the user that you would be happy to sell it to them, but you cannot. If they are not of age, tell them to get out of the bar. If they are of age but cannot afford the cocktails, tell them that you are sorry and that this is awkward but they do not have enough.

20. Ask the user for their age. Then, ask them if they drive a car or not. If the user drives a car before being 18, let them know they are breaking the law. Otherwise, ask them how long ago they got their license. If their license would have been taken before they were 18, ask them how that was possible.

21. **Extra food for thought small challenge:** Create a program that asks the user for their first name. Then, if they have more than two "A"s in their name print "That's plenty of A's you got there". If they do not, say nothing. **Note:** You are not supposed to solve this exercise using a cycle (if you by chance know them).

22. **Another Extra food for thought small challenge:** Ask the user to insert a sentence. Then, if the sentence has more than 10 vowels, print its first two letters. Otherwise, print the last two **Note:** You are not supposed to solve this exercise using a cycle (if you by chance know them).

7 Loops: While and for loops

7.1 While loops

What's the difference between if and while?

while is, in its logic, very similar to an **if**...

- After **while**, there is a Boolean expression. It can be a relational expression, or logic expression (they both return either True or False).
- Just like **if**, whenever there is an indentation (tab) after **while**, this shows that, that line belongs to **while** (inside while).
- We execute that line only if the Boolean expression is True.
- Until here, it is exactly the same as **if**.
- The difference is, after we execute the code inside **while**, instead of continuing the code, we go back and check if the Boolean expression still True.
- If it is still True, we execute the code inside **while** again, and then check the Boolean expression again. We continue this until the Boolean expression is False.

```
while <boolean expression>:
      <block of instructions>
```

Notes on while loops...

- When using **if** or **while**, if you are doing 2 comparisons, never ever write them like this:

if department == "management" or "software":

This doesn't make sense because logic expressions expect to have 2 Boolean values on each side. "software" is not a Boolean expression. The correct way is:

if department == "management" or department == "software":

- When comparing be careful to write == instead of =
  Remember, == means "is it equal?", = means "is"
- Don't forget to increase or decrease or change your variable in the Boolean expression, unless you want your loop to turn forever. (like x + = 1)
- Even if you exit the loop because the Boolean expression is False, be careful about the last value of x. For example in this case, the last value of x is 3.

- Don't forget the : after the Boolean expression!
- Don't forget the indentation if you want an line to be "inside while".

```
x = 1
while x < 3:
    # i'm inside
    print(x)
    x += 1

# i'm outside
print(x)
```

Prints
1
2

Prints
3

7.2 Obtaining numerical input from the user: the is.digit() function

7.2.1 the is.digit() function

From now on, if we want a number from the user (either int or float), we will make sure we get what we want! And we will ask for it again and again until we get it!

.isdigit() is literally asking the question, "is every character of this string a digit?". It returns either True or False.

We will use this function with while to keep asking user to insert a number again and again.

In the case of float, we have a small trick because "." is not a number. That's why .isdigit() will return False with a float. How do we solve it?

x.replace(".","1")

Reminder:

<string>.replace("old","new") does NOT change <string> permanently!

So, here we replace any existing "." with "1" (or any other number). And we do the comparison with our "fake" number because the value of <string> won't change actually.

For example "4.5" becomes "415" temporarily.

"4.5".isdigit() returns False

"415".isdigit() returns True

```
x = "4.5"
if x.replace(".", "1").isdigit():
    print(x)

# 4.5 is printed on console!
```

7.2.2 Detecting an integer, a float and string/bool

Here's how we detect an integer (if), float (elif) and string/Boolean (else):

```
x = input('Give me an input!')
if x.isdigit():
    print('You gave me an integer number')
elif x.replace(".", "1").isdigit():
    print("you gave me a float!")
else:
    print("You have not given me a number...")
```

7.3 Assuring user input is numerical

Here's how we keep asking for an integer or float from the user until we get it. Take a moment to understand these steps:

```
x = input("Insert an int or float:")
while not x.replace(".","1").isdigit():
    x = input("Nope. Try again:")
print("You did it!")
```

1. **not x.replace(".", "1").isdigit()** will be True if user writes some letters.

2. That means, we will execute the second **input(...)**

3. We will keep asking until the Boolean expression is False.

- Which means until **not x.replace(".", "1").isdigit()** is False

- Which means until **x.replace(".", "1").isdigit()** is True.

7.4 For loops

Why do we use a for loop/cycle? and what's the difference between if, while and for?

if is a conditional instruction in which the inside code (don't forget the indentation which means it is in the following lines and there is a tab at the beginning of the line of code) only runs 1 time if the Boolean expression is True.

while works in a very similar way, because we only execute the code inside the **while**,

when the Boolean expression is True.

The difference between **if** and **while** is, after the inside code is executed, the Boolean expression is checked again. If it yields True the code inside is executed again. Once more, upon that execution the Boolean expression is checked again. This cycle (of checking the boolean variable and running the inside code if it is True) only ends when the Boolean expression changes to False.

In while loops, the number of iterations (i.e., the number of times the inside code is executed) depends on how long it takes the Boolean condition to turn False.

so in for loops...

We use **for**, when we already know the number of iterations, it does not depend on a condition. Notice that unlike **if** and **while**, we don't have Boolean expressions in **for**.

```
for <variable> in <sequence>:
        <block of instructions>
```

We stop the loop/cycle once we reach the limit of the <sequence>.

- <block of instructions> is repeated for a number of times equal to the length of <sequence>

- <sequence> can be:

  - A string (that we defined or received from the user)

  - A sequence generated by **range(start, stop, step)** function

    * Just like string slicing with indexes, first number is included, second one is excluded!

      range(start, stop, step)

      You don't need to use all 3 elements: (by default, start is 0 and step is 1)

range(1,5) means 1, 2, 3, 4 (start, stop)
range(5) means 0, 1, 2, 3, 4 (stop)
range(1,10,2) means 1, 3, 5, 7, 9 (start, stop, step)

- At each iteration, <variable> has a different value from <sequence>
  At iteration 1: <sequence>[0] , at iteration 2: <sequence>[1] , . . .

  So...

- If it's a string like "bye", variable will be "b" in the 1st iteration, "y" in the 2nd, "e" in the 3rd.

- If it's range(1,3), variable will be 1 in the 1st iteration, 2 in the 2nd iteration.

We named the variables i and letter, but you don't have to. We usually use i when it refers to index or iteration. However, you can choose the variable name however you wish (Following the variable name rules, we discussed previously).

```
for i in range(1, 5):
    print(i)

# prints:
# 1
# 2
# 3
# 4

for letter in "hello":
    print(letter)

# prints:
# h
# e
# l
# l
# o
```

7.5 Exercises

These exercises are to be solved using loops. Do not use lists if you know them just yet!

1. Ask the user for a positive number, x . Print all numbers from 2 to x.

2. Ask the user to insert an integer number, n. Print all the numbers from 1 to n (including n) that begin with the digit 1. For instance, if the user inserts the number 10, 1 and 10 should be printed.

3. Ask the user for their age. If they are younger than 30, print all the even numbers from 1 to their age. Otherwise, print all the odd numbers from 1 to their age.

4. Ask the user to insert an integer number from 1 to 10. Insist that the number is within that interval (i.e., if the user inserts a string, or a number outside of that interval, the program repeats the request of the number and does not advance until a valid number is inserted). Print the multiplication table of that number until 10. For example, if the user inserts 4 print 4 x 1 = 4, 4 x 2 = 8 ... 4 x 10 = 40.

5. Ask the user to insert a word that starts with the letter 'A'. The program does not advance until the word begins with the letter 'A'. Print the word in console k times, where k is the size of the word. For instance, the word "Ant" is printed 3 times.

6. Ask the user to insert a positive integer number, n. The program does not advance until a positive number is inserted. Print on console the sum of all numbers from 1 to n (including n).

   - **TIP:** we can have more than one auxiliary variable. The only difference between this exercise and exercise 2) is that instead of printing all the numbers from 1 to n we add them to a variable created outside of the loop called, for example, result.

7. Write a program that asks the user if they would like to buy 1 - ice cream, 2 - cheesecake or 3 - peanut butter snacks. The program does not advance until a valid choice is inserted. Print the total cost of user's purchase with tax (+6%) knowing that:

   ice cream costs 1 euro without tax.
   cheesecake costs 4 euros **with** tax.
   peanut butter snacks cost 2 euros without tax.

8. Write a program that asks the user for their computation I final expected grade. The program does not continue until a valid grade (in the interval [0-20]) is inserted. If the grade inserted is below 18, let the user they can aim higher!

9. Ask the user for their full name. Print on console how many vowels are in it. For example, if the name inserted is 'Vendetta Rosenfeld', the number 6 is printed on console.

10. Write a program that receives two integers from the user, a and b. a must be positive and b must be from 1-10. The program does not advance until these conditions are met. Display the first a multiples of b. For example, if b is 2 and a is 8, we would want to print the first 8 multiples of 2 so: 2 4 6 8 10 12 14 16.

11. Create a program that asks the user to pick a username. The program does not advance until the user picks a username that has at least 6 characters and the special character "_".

12. Ask the user for a number, n. This number corresponds to how many numbers they would like to insert and therefore the program cannot advance until the user inserts a positive number. Allow them to insert n numbers. Print the sum of all the numbers they inserted. For instance, if the user picks n = 2 and then inserts 10 and 6, the number 16 should be printed on console.

13. Create a program that receives numbers from the user. The program stops when the user inserts -1. At the end of the program print how many numbers were inserted.

14. Ask the user to insert a word. Then tell the user to select 1 if they want to print the string inverted and 2 if they want to know how many vowels are in the string.

15. Ask the user for their two Computation I mini-tests grades. The program doesn't continue until valid grades are inserted. Calculate the average of the two grades.

16. Create a program that receives three numbers (a, b, c). All of these numbers must be positive integers, the program does not continue until they are. Check how many numbers between a and b are divisible by c.

17. Create a program that receives numbers from the user. The program stops when the user inserts "q". At the end of the program print the average of the numbers

that were inserted.

18. Ask the user for their birthday in the following format dd/mm/yyyy. The program doesnt advance until the user inserts a valid date. Print the users age in january 1st 2024

19. Ask the user to guess the name of a programming language. the program allows the user to keep guessing until they get the correct answer right. Count how many tries it took the user to guess the right answer. For the sake of this exercise, assume that Python is the right answer.

20. Ask the user to guess a number from 1 to 10. The winning number can be a number of your choice. Give the user 3 tries to guess the number right. When they do, congradulate them and inform them that they got the right answer. If they tried 3 times and failed, inform them that they lost and what the winning number was.

21. Ask the users how many children they wish to have, x. Then, ask the user for the names of the x children that they want to have. Print on console the total amount of vowels used in the names

22. Ask the user for three numbers: a, b and c. The program doesn't terminate until the user inserts numbers so that a $<$ b $<$ c. Then, print on console the result of $a \times b \times c$.

8 Lists: Part II

8.1 Traversing Lists

- Lists can be traversed with a for loop. This creates a loop with the same number of iterations as the length of the list, where the loop's variable, is, at each iteration, a specific value of the list.

```
for i in my_list:
    # Do something with i
```

- However, it is very important to note that i is a copy of the list's values, not the original! This means that any changes to it will not affect the original list! If you want to change a list's values with a for loop, you MUST iterate not over the list, but its indexes, and change it through the indexes.

```
for i in range(len(my_list)):
    my_list[i] = # Something
```

8.2 List Comprehensions

- List Comprehensions are, essentially, a mixture of a for loop and the creation of a list. They execute a loop for a defined amount of iterations, and, for each one, place a specific value inside a new list. Thus, they can be used to create new lists.

```
my_list = [i for i in range(3]
print(i)
# Output
>>> [0,1,2]
```

- The recommended way to read a list comprehension is to first look at the for instruction, and then at what is to the left of it (which is what is placed on the new list). In the previous example, the loop runs three times, with i having the values of 0, 1 and 2. Then, each iteration places the value to the left, i in the new list. So, the new list will be [0, 1, 2].

- The value on the left can also be more complex. In fact, anything that qualifies as an expression (be it a variable, a number or the result of an operation) can be used. For example, the code below creates a new list composed of the values of another one, to the power of 2.

```
new_list = [i ** 2 for i in my_list]
```

- It is important to note that list comprehensions can always be replaced by an equivalent for loop where you add elements to an empty list each iteration. However, they are a quick way to create lists, which can be very useful!

8.3 Exercises

1. Using list comprehensions, create a list with all the numbers from 6 to 16, including those numbers and print it.

2. Create a list of 6 values of your choice. Using a for loop, sum all the odd values of the list.

3. Create a list of 10 values of your choice. Using a for loop, sum all the values of the list that begin with a two and are odd.

4. Ask the user how many siblings they have. Then, allow the user to insert the name of each sibling. Store the names in a list. At the end, return the longest name.

5. Ask the user how many books that have that are late to be returned to the library. Next, ask the user for the names of the books and store that information in a list. Then ask the user how many days past return date is each book, storing the values in a list. Then, assuming each delay day per book costs 2 euros in fines, print on console the following information:

    - The total fine the user has to pay
    - The name of the most expensive book in terms of fines

6. Using list comprehensions, receive 5 integers from the user. Print the minimum number in the list.

7. Write a program that:

    - Asks the user how many siblings (brother, sister) they have. The program does not advance until a positive integer or 0 is inserted.
    - if they have 0 sibling, print on the console, "So you don't ever have to share!".
    - If they have 1 or more than 1 sibling, let the user create a list filled with the ages of them and their siblings.
    - Print on the console the sum and the average age of all the siblings (including user).
    - Print on the console how many "adult" (18 or higher) children that family has.

8. Write a program that:

- Asks the user the names of their 5 friends and store those names in a list.

- Create a new list with the people whose name starts with "B". Print the new list.

9. Ask the user to insert their top 10 favourite games and store that information in a list. Count how many Assassin's creed games are in the list.

10. Ask the user to insert 10 numbers that are between 10 and 74. For each number, do not allow the user to insert a value that does not meet that criteria. Then print on console how many numbers that are divisible by 3 were inserted by the user.

11. Consider the following two lists:

    games = ["Red Dead Redemption II", "Cyberpunk", "Madden 24", "The Witcher 3", "Assassin's Creed II"]

    costs = [40, 40, 70, 25, 10]

    Print the name of the cheapest game

12. Ask the user how many classes they took last semester. Then, ask the user for the names of the classes and the obtained grades, storing the information in two lists (called classes and grades). Print the names of all the classes where the user obtained a grade that is better than 17.

13. Ask a user for two numbers, a and b. Using a comprehension list, create a list with all the numbers from a to b (including b).

14. Ask the user to insert the grades of the five classes they took in the first semester of last year. Then, ask the user to insert the grades of the five classes they took in the second semester of last year. Print on console the following information:

    - A list with all the grades

    - The average of the first semester

    - The average of the second semester

15. Create a list with 10 values of your choice. Modify the value of the first even number to 74. Make sure you **only** modify the first even number and not the remaining ones.

16. Write a program that asks the user to insert the values for two lists (of size 4). Return the factorial of the number of values they have in common. For example the list [1,2, 3, 4] and [3, 4, 5, 6] have 2 numbers in common, so the program prints the factorial of two which is 1 * 2 = 2.

17. Write a program that takes 3 values as input and stores them in a list. Return the sum of the two highest values.

18. Ask the user for 8 numbers of their choice. Saving the values in a list, print the product of all the values of the list that are higher than the list's average.

19. Create a list with 4 values of your choice. Then, ask the user to insert 4 values of their choice. Create a new list that contains all the values from both lists. Then, filter out repetitions from the list and find the amount of numbers that end with a 0.

20. Create two lists, a and b, with 6 values of your choice, each. Then, ask the user to insert a number, i, from 0 to 5 (including 5). The program does not continue until a valid number is inserted. Then, swap the values in the ith position of the two lists. For instance, consider the lists a = ["one", 3, 6, 10, "potato", 20], b = [1, 2, 3, 4, 5, 6]. If the user inserts i = 4, a becomes ["one", 3, 6, 10, 5, 20] and b becomes [1, 2, 3, 4, "potato, 6].

9 Matrices and nested for loops

9.1 Matrices

- Matrices are simply lists where each element is another list, thus giving them a two dimensional structure. We can imagine them as being matrices similar to those in Algebra: each of the inner lists represents a row. The next example shows how a matrix would be represented in Python.

$$\begin{matrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 6 & 9 \end{matrix}$$

```
my_matrix = [[1,2,3], [4,5,6], [7,8,9]]
```

- As matrices are lists, you can use indexes to access its elements. Since each of them is a list (meaning, it's a row), you can use another index to access an individual

element. This means that any element can be accessed with the notation matrix $[i][j]$, where i is the row of the element and j is the column. Remember that indexes start at 0!

```
print(my_matrix[1][2])
# Output
>>> 6
```

- Normal lists are one-dimensional, and thus require one loop to traverse. As a matrix is a two-dimensional structure, we require the **usage of two loops** to traverse them. Essentially, we need one loop to iterate over the rows, and another for the columns. Do remember that, as with lists, we need to access the indexes if we want to **change** values.

- Keep in mind the number of rows is always equal to the length of the outer list, the matrix object itself, and the number of columns is equal to the length of one of the inner lists (one of the rows). You can use this information to directly access specific portions of it, if needed.

- The most simple way to understand matrices (i.e., multi-dimensional lists) is to see them as exactly what they are: lists of lists! That is, we know that lists are sequences that contain several variables. Matrices are simply lists where each veriable is a separate list. Thus, its intuitive that we would need a for loop to iterate through the matrix itself and then another for loop to iterate through each list inside of the matrix. The figure below shows two different ways in which a nested for loop can be used to iterate through a matrix.

```
for row in my_matrix:
    for element in row:
        # Do something with element
for i in range(len(my_matrix)):
    for j in range(len(my_matrix[i])):
        matrix[i][j] = # Do something
```

9.2 A more in depth look into Nested for loops

Nested for loops are easily understood when we take a second to decompose them and simulate their iterations in a step by step fashion.

They are composed of a loop that's inside of a loop. Thus, the first loop is commonly known as the outer loop and the second loop is commonly known as the inner loop (as it is inside of the outer loop).

```
for i in range(2):
    for j in range(2):
        print(f'I print i = {i} and j = {j}')
```

As we can see in the code above, the outer loop is usually indexed using i as variable name while the inner loop uses j. Naturally, variable naming is entirely up to you and does not actually matter. However, we recommend using common nomenclature as it makes your consultation of external materials easier.

Executing the above code prints the following on console:

"I print i = 0 and j = 0"
"I print i = 0 and j = 1"
"I print i = 1 and j = 0"
"I print i = 1 and j = 1"

The logic behind this output is the following:

1. the outer (i) loop begins with i = 0.

2. Since the block of instructions to be executed inside of the loop is another loop (the j loop) we begin the inner for with j = 0.

3. The instructions of the j loop are executed and therefore "I print i = 0 and j = 0" is printed on console

4. As it occurs in every for loop, once the execution of the block of instructions is done, we move to the next iteration and so j becomes 1

5. Once again, the j loop instruction block is executed and "I print i = 0 and j = 1" is printed on console.

6. Completing the j loop means we have completed the block of instructions associated with the i for loop. Thus, it is time to advance to the next iteration in the i loop, making i become 1.

7. With i at 1 the block of instructions associated with the i loop is once again executed. Surprise! Since it's a loop we once again set j = 0 and enter it's instruction block

8. The execution of the j instruction block prints "I print i = 1 and j = 0" on console.

9. Once again, finishing the execution of the j instruction block means that it's time to move to the next iteration of j (making j = 1) and execute the j instruction block once more.

10. The j instruction block now prints "I print i = 1 and j = 1" on console.

11. Finishing the j instruction block means we are done with the j inner loop once more and we move on to the next iteration of the i loop. At this point, since i only has two iterations, we are done executing our code.

Thus, we can conclude that the j loop will be executed in its entirety i times! This makes total sense because it is part of the i loop's instruction block! Hence, we will execute the j loop from the beginning at each iteration of i.

9.3 Exercises

1. Considering the matrix below, answer the questions: M = [[5, 42, 3, 1], [3, 1, 5, 2], [3, 94, 3, 1], [3, 5, 3, 2]]

   - Print 94 using indexes.

   - Update the values of the elements on the odd valued columns to 0 (column with index 1 and 3).

   - Print the numbers in the one diagonal of the square matrix, the one from the upper left corner to the lower right corner

2. Create a program that asks the user how many students they would like to represent. Then, ask the user for the following information on each student: their name, age, student number and expected Computation I grade. Store the information in a matrix (list of students, having each student's information as a list on its own). Finally, print the average expected grade from the students.

3. Ask the user how many employees their company has. Then, individually ask how much each employee is making per month and how many years they have been in a company. Give the employees that have been in the company for over 6 years a 5% raise and print the resulting matrix on console.

4. Create the matrix below in your code M = [[5, 42, 3, 1], [3, 1, 5, 2], [3, 94, 3, 1], [3, 5, 3, 2]]

   Then, add to each row a value that corresponds to the sum of all the values of that row. For instance, the row [5, 42, 3, 1] becomes [5, 42, 3, 1, 51] and so on and so fourth.

5. Create a program that asks the user how many columns and how many rows they want the matrix to have. Then, allow the user to fill out the matrix with integer values only, if a float or any other value is inserted the program repeats the request. Then, print on console the product of all the odd values in the matrix.

6. Ask the user how many spices they are going to add to their brisket. Then, allow the user to fill out the information of the spices in a matrix, storing for each spice its name, the amount (in grams) that are to be used and the price per 100 gr. Print the name of the most used (in grams) spice as well as the spice that will be the most costly.

7. Allow the user to fill in a 3 x 3 matrix with numbers. Find the smallest number.

8. Allow the user to fill in a 3 x 3 matrix with numbers. If the biggest number is divisible by the smallest value, inform the use that they have inserted a lovely matrix. If not, tell them you don't like it.

9. Using nested for loops, print the following pattern on console:

   0
   1 1
   2 2 2
   3 3 3 3

10. Using nested for loops, print the following pattern on console:
    *
    * *
    * * *
    * * * *
    * * * * *

```
* * * *
* * *
* *
*
```

11. Allow the user to fill a 3 x 3 matrix with floats only. If the user inserts anything that isn't a float, the program keeps insisting upon input request. Then, sum all the values in the matrix that begin with 2.

12. Allow the user to fill a 4 x 4 matrix with integer values of their choice. Add a final row that contains the summation of all the values of each column. That is, in position 0, the row will have the sum of all the values of column 0 and so on and so fourth.

13. Ask the user how many friends they have. Then, for each friend ask the use about the name of the friend and age of their mother and father. And store the information accordingly in a matrix. Finally, print the name of the friend who has the oldest father.

14. Allow the user to fill a 4 x 4 matrix with numbers of their choice. Print the sum of all the even column values, the sum of all the odd column values and the product of all matching column and row indexes values (e.g., mat[0][0] or mat[1][1])

15. Create a 4x4 matrix with random values between 1 and 50. Then:

    - Print the sum of all the elements in each row.
    - Print the maximum element from each column.
    - Replace all even values in the matrix with -1 and print the updated matrix.

    Hint: Use the `random` library in Python. You can generate a random integer between 1 and 50 using `random.randint(1, 50)`.

16. Ask the user to input dimensions (rows and columns) for a matrix, and then allow them to fill the matrix with integer values. Calculate the following:

    - The average of each row and store these averages in a separate list.
    - Print the row that has the highest average.

17. Allow the user to fill a 5x5 matrix with numbers of their choice. Then, modify the matrix by:

    - Swapping the values in the first and last rows.
    - Swapping the values in the first and last columns.
    - Print the resulting matrix.

18. Create a program that initializes the matrix below:

$$M = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]$$

Then:

- Replace the middle element with the sum of the elements on the main diagonal.
- Multiply each element in the last row by 3.
- Print the updated matrix.

19. Ask the user to fill in a 3x3 matrix with integer values. Then:

- Calculate the sum of the elements on the secondary diagonal (from the top-right to bottom-left).
- Replace each element on the secondary diagonal with -1 and print the modified matrix.

20. Create a matrix representing monthly sales figures for a company, with each row representing a salesperson and each column representing a month (e.g., a 3x4 matrix). Allow the user to fill in each value. Then:

- Calculate the total sales for each salesperson and print them.
- Find and print the highest sales value along with the salesperson and month associated with it.

10 Functions

10.1 Function creation and calling

What are Functions?

Functions are blocks of code we define once but can use multiple times. They are used to prevent repeating code and to organize the structure of our program.

More specifically, they are blocks of code that are only executed when you call them (i.e., when you call the function)

Functions **may** take an input (but don't have to) and **may** have an output (but, again, don't have to!)

How do we create a function?

def <name of the function>(<sequence of formal parameters>):
    <block of instructions>

The <sequence of formal parameters> consists in the inputs that the function will receive. As previously stated we **don't** have to have inputs. When we don't, we just write def <name of function>():

The <block of instructions> consists in the block of code that will be executed when we call the function.

In order to **call** the function we must write:
<name of the function>(<sequence of actual parameters>)

- Note how **when we are creating the function**, the name of the theoretical inputs is **"formal parameters"** while **when we call the function** the name of the actual input that we are giving to the function are called **"actual parameters"**. This is because when we create a function we are telling the computer "You will receive a set of parameters with a theoretical name of *insert names here*. Then you will use these theoretical parameters to perform calculations/operations. We therefore write the code in a general way, without ever knowing the actual values that the program will receive. Later on, when we call the function we give "actual" values to the "theoretical" ones we wrote the code with.

If we wish to have the function have an output, we need to use the **return** keyword followed by what variable we wish to return.

Generally speaking, when we call a function we are running its <block of instructions> in a different space in memory. That is why if we want to have access to a variable outside of the function we must use the **return** keyword at the **end** of the function in order to return it. You will learn more about this special memory space during classes. For now, remember that **generally speaking** if you want to access a variable outside of a function, we must return it.

Function creation and calling example:

- An example of a function with **no** input and **no** output can be found below:

  ```
  def printing_hello():
      print("Hello!")
  ```

  Whenever this function is called, we will print "hello" in the console.
  To call this function all we need to do is write printing_hello() on console/our code.

- An example of a function with **two input variables** and **one** output variable can be found below:

```
def two_sum(a, b):
    result = a+b
    return result
```

This function **must** receive two values as input when called. It will return the sum of the two variables given as input.

For example, in order to call this function we could write res = two_sum(10, 12) on console/our code. Note that since this function has an output (i.e., returns something) we **must save** its result in a variable in order to be able to access it.

Additionally, we could call the function and pass previously existing variables to it as input. Imagine we created variable pocket_money = 10 and a variable wallet_money = 30. We can sum the two values by calling total = two_sum(pocket_money , wallet_money)

Note that when we created the function we gave it two formal parameters: a and b while when we called the function the actual parameters were pocket_money and wallet_money (so 10 and 30, respectively).

- An example of a function that receives **one** input and has **no** output can be found below:

```
def message_printer(message):
    print(message)
```

This function receives a message as input and prints it on console! We could call it by writing message_printer("Weddings are like funerals with cake") which would lead to the string "Weddings are like funerals with cake" to be printed on console.

- **IMPORTANT NOTE:** When we say a function has no output, we mean it doesn't return a variable. This doesn't mean that it doesn't perform an action. The last function explained above is a perfect example of this. It doesn't return a variable value but it performs a specific action (printing on console) that is needed from it.

10.2 Function rules

- The order in which we create the functions does not matter. What matters is the order in which we **call** them. Think about creating a function as making that reusable piece of code available to you for whenever you need it. However, it is only executed when you **call** it.

  - That being said, one thing that is important is to **create the function before calling it**! You cannot call a function that doesn't exist! Once created, the function is only executed when called and that's why the order in which you create the functions does not matter but the order in which you call them does!

- Once you create a function you can call it as many times as you want! They are reusable pieces of code that are at your disposal without a numbered limitation of usage.
- We can create functions inside of functions. We will talk about this later on, for now, just remember that it is possible.
- There can **only be one return** statement in a function. You cannot have multiple returns
- the return keyword must be the last line of code inside the function. Everything **after the return is considered dead code** and is not ever executed or accessible!
- If you call a function that has a return without saving the result in a variable, you will not be able to use that result later on in your code.
- The name of the formal parameters does **not** have to be the same as the name of the variables you will use to call the functions with!

10.3 Exercises

1. Create a function that asks the user to fill out a 4 x 4 matrix and calculates the average values in the matrix.

2. Create a function that receives two numbers and returns their sum divided by two.

3. Create a function that receives two lists as input and returns a list of the uncommon elements between the two lists.
4. Create a function that receives 3 values and returns them in a list.

5. Create a function that receives two matrices and creates a third matrix whose elements correspond to the multiplication of the elements of the other two matrices.

6. Create a function that receives a list and multiplies all of its odd index values by two.

7. Create a function that receives a list as input and returns the list inverted.

8. Create a function that receives a matrix and a number, n. This function changes all the values in column n to the value 20. For example, if the user inserts [[1,2,3], [4,5,6]] and 1, the function should return [[1,20,3], [4,20,6]]

9. Create a function that receives a matrix m and a number n, if the number n is odd the function returns a list with the average values of each row. If the number n is even, the function returns a list with the averages values of each column.

10. Create a function that receives a number, n, and returns its factorial divided by the number of numbers between 1 and n.

11. Create a function that receives a matrix, m and returns a list with the maximum value of each row.

12. Create a function that receives a list of names and returns the list uppecased.

13. Create a function that receives a matrix and checks if it's an identity matrix. A matrix is an identity matrix (note that an identity matrix has 1 in the main diagonal and 0s elsewhere).

14. Create a function that receives a list of strings and a string and prints the receives string as many times as it is repeated in the list.

15. Create a function that receives a sentence and prints how many words in that sentence begin with a vowel.

16. Create a function that takes as input a sentence and calculates how many vowels exist in each word and returns this information.

17. Create a function that receives a string as input and returns the number of words in the string.

18. Create a function that receives a list of strings and prints the biggest string, the smallest string and the average size of the strings in the list.

19. Create a function that receives two matrices and calculates the dot product between them.

20. Create a function that receives two integer numbers. The function is able to identify the largest number of the two and return a list containing all the numbers from the largest to the smallest.

Good work! ☺

In case of any doubts

Prof. Leonardo Vanneschi	lvanneschi@novaims.unl.pt
Prof. Liah Rosenfeld	lrosenfeld@novaims.unl.pt
Prof. Samuel Santos	samuel.santos@novaims.unl.pt
Prof. Diogo Rasteiro	drasteiro@novaims.unl.pt